Министерство образования Республики Беларусь
Учреждение образования
Белорусский государственный университет информатики и радиоэлектроники

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

# ОТЧЁТ
по курсу
«Естественно-языковой интерфейс интеллектуальных систем»

Лабораторная работа №3
«Синтаксический анализ текстов естественного языка.»

Выполнили студенты гр. 321701: Политыко И.А.
Лукашов В.А.

Проверил: Крапивин Ю.Б.

Минск 2026

## Цель работы

Освоить принципы разработки прикладных сервисных программ для решения задачи анализа текста естественного языка.

## Задачи лабораторной работы

1. Познакомиться с назначением, структурой и функциональностью, предоставляемой базовым ЛП для решения задачи автоматического синтаксического анализа ТЕЯ.
2. Закрепить навыки программирования при решении задач автоматической обработки ТЕЯ.

## Задание

**Вариант 2:**

- входные данные – текст заданного естественного языка;
- выходные данные – структуры, полученные при проведении автоматического синтаксического анализа предложений входного текста согласно варианта задания;
- взаимодействие с пользователем посредствам графического интерфейса (интерфейс должен быть интуитивно-понятным и дружественным пользователю);
- наличие системы средств помощи пользователю;
- обеспечение возможности построения, сохранения, просмотра, редактирования, документирования автоматически получаемого результата либо заданной его части;

**Поддерживаемые форматы входных данных:** TXT, RTF
**Язык текста:** Русский

**Используемые библиотеки**

1. FastAPI — библиотека Python для создания высокопроизводительных API. Используется для разработки сервера обработки запросов и обеспечения взаимодействия между фронтендом и бэкендом.

2. Pydantic — библиотека Python для валидации данных. Позволяет создавать строгие модели сущностей, что гарантирует корректность поступающей информации и упрощает обработку ошибок.

3. pymorphy3 — библиотека Python для морфологического анализа русского языка. Используется для лемматизации слов, а также определения их части речи, падежа, числа и рода.

4. razdel — библиотека Python для токенизации русского текста. Используется для качественного разделения исходного текста на отдельные предложения и токены.

5. striprtf — библиотека Python для обработки документов в формате RTF. Позволяет извлекать чистый текст из файлов данной разметки для последующего синтаксического анализа.

6. SQLAlchemy — библиотека Python для работы с базами данных через объектно-реляционное отображение (ORM). Используется для организации хранения документов, токенов и синтаксических связей в базе данных SQLite.

7. spaCy — библиотека Python для продвинутого анализа естественного языка. Применяется для построения деревьев зависимостей, что позволяет определять иерархические связи между словами в предложениях.

8. React — JavaScript-библиотека для создания пользовательских интерфейсов. Используется для реализации интерактивной панели управления и динамической визуализации результатов анализа.

9. TypeScript — строго типизированный язык программирования, расширяющий возможности JavaScript. Обеспечивает высокую надежность кода фронтенда и упрощает поддержку проекта за счет статической типизации.

10. Vite — современный инструмент сборки фронтенда. Используется для быстрой компиляции проекта и оптимизации работы веб-интерфейса.

## Интерфейс

Интерфейс приложения включает следующие элементы:

• **Окно анализа:** пользователь загружает, удаляет, просматривает и редактирует тексты, а также управляет процессом синтаксического анализа файлов TXT и RTF.

• **Окно результатов:** отображает морфологические признаки токенов (лемма, часть речи, грамматика) и общую статистику по документу.

• **Окно связей:** содержит таблицу синтаксических отношений между словами в предложениях с цветовой маркировкой типов связей.

• **Окно деревьев:** обеспечивает интерактивную визуализацию деревьев зависимостей и деревьев составляющих.

• **Окно справки:** содержит документацию с используемыми терминами, описанием API и возможностями системы.

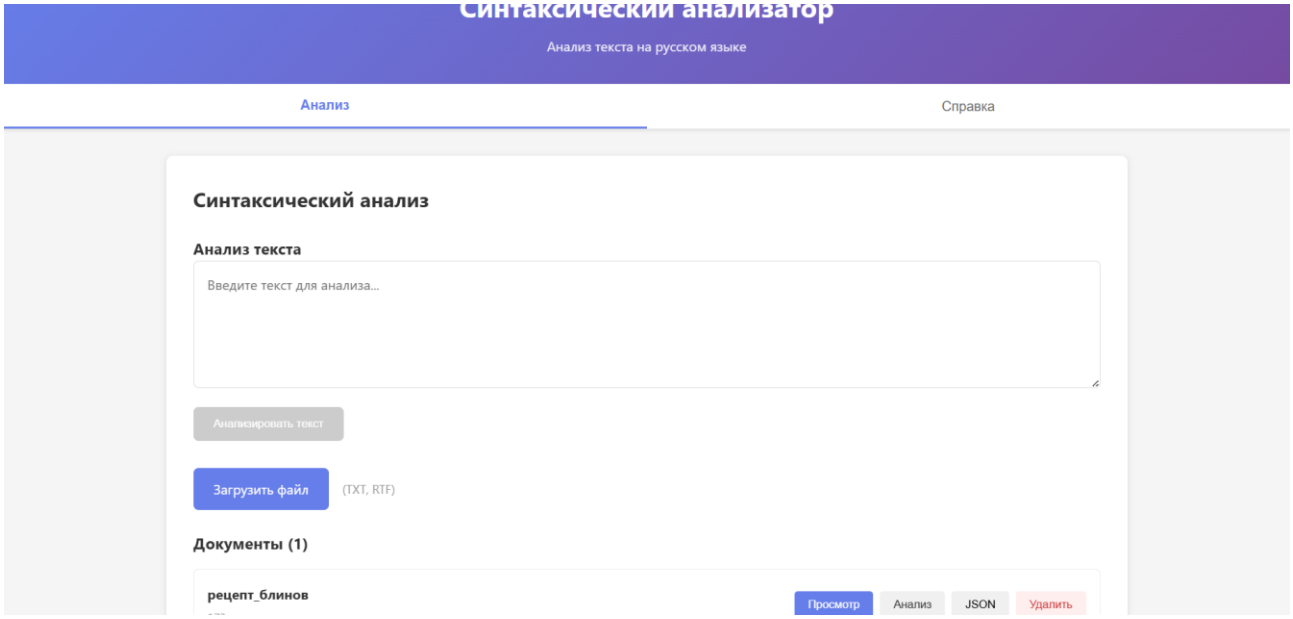


***Рисунок 1:*** *Интерфейс анализа*

**Результаты анализа**

Показать связи

**Статистика**

| Общая информация | |
|---|---|
| Предложений: | 19 |
| Слов: | 204 |
| Время анализа: | 39 мс |

| Части речи | |
|---|---|
| существительное | 65 |
| пунктуация | 46 |
| прилагательное | 28 |
| глагол | 21 |
| союз | 11 |

| Члены предложения | |
|---|---|
| дополнение | 53 |
| определение | 30 |
| сказуемое | 25 |
| подлежащее | 15 |
| обстоятельство | 7 |

**Предложение 1** 18 слов

*"Домашние блины на Блины — традиционное русское блюдо, которое готовят из муки, яиц и молока."*

**Синтаксические связи:**

| № | От кого | Куда | Тип связи |
|---|---|---|---|
| 1 | блюдо | готовят | Подлежащее → Сказуемое |
| 2 | готовят | муки | Сказуемое → Дополнение |
| 3 | готовят | яиц | Сказуемое → Дополнение |
| 4 | готовят | молока | Сказуемое → Дополнение |

← 1/2 →

***Рисунок 2:*** *Интерфейс результатов анализа*

**Результаты анализа**

Скрыть связи

**Синтаксические связи**

| № | От кого | Куда | Тип связи |
|---|---|---|---|
| 1 | блюдо | готовят | Подлежащее → Сказуемое |
| 2 | готовят | муки | Сказуемое → Дополнение |
| 3 | готовят | яиц | Сказуемое → Дополнение |
| 4 | готовят | молока | Сказуемое → Дополнение |
| 5 | Домашние | блины | Определение → Существительное |
| 6 | русское | блюдо | Определение → Существительное |
| 7 | Они | получаются | Подлежащее → Сказуемое |
| 8 | рецепт | поможет | Подлежащее → Сказуемое |
| 9 | рецепт | испечь | Подлежащее → Сказуемое |
| 10 | поможет | вам | Сказуемое → Дополнение |
| 11 | поможет | блины | Сказуемое → Дополнение |
| 12 | испечь | блины | Сказуемое → Дополнение |

***Рисунок 3:*** *Интерфейс связей синтаксического анализа*

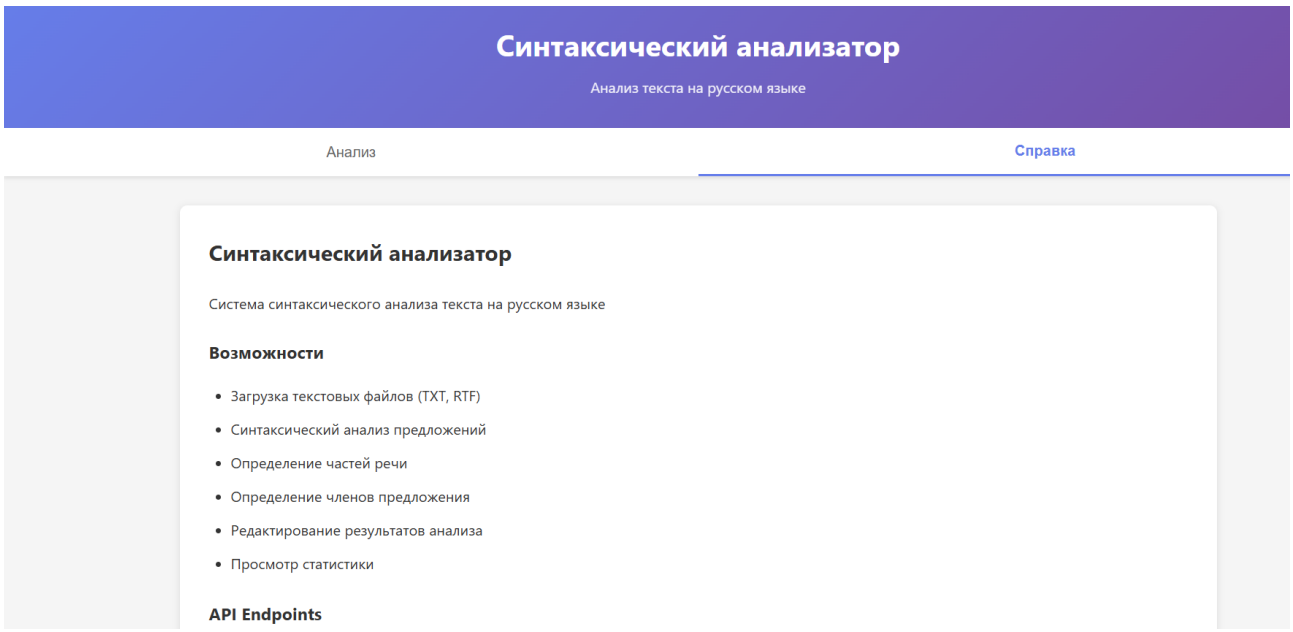


***Рисунок 4:*** *Интерфейс панели справки*

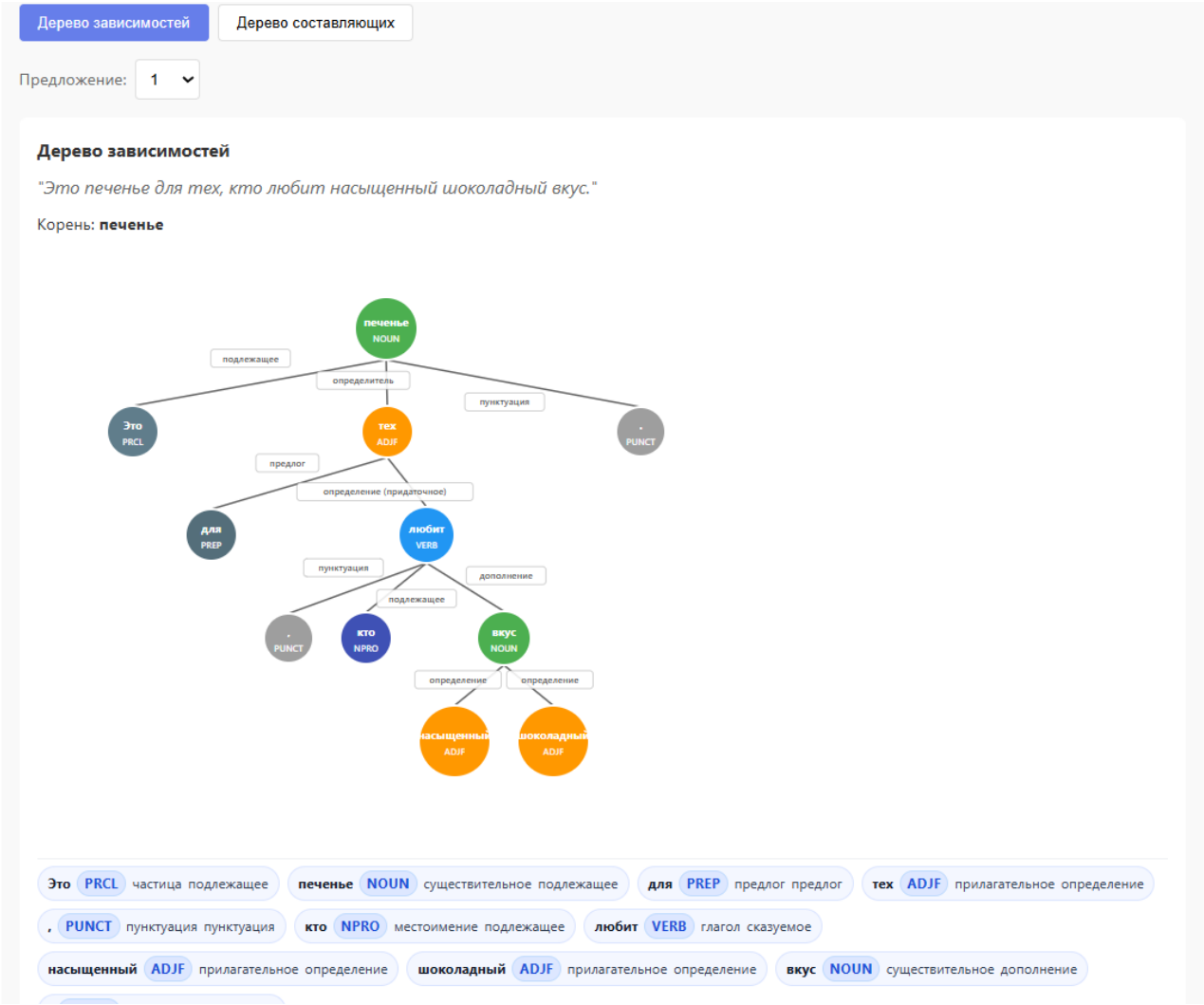


***Рисунок 5:*** *Интерфейс дерева зависимости*

# Структура хранения данных

Данные хранятся в реляционной базе данных SQLite.
В таблице **documents** (Коллекция документов) хранятся:

- **Уникальный идентификатор** (id): UUID документа;
- **Заголовок документа** (title): Название документа;
- **Текст докумета** (content): Полное текстовое содержимое;
- **Поля метаданных**, где каждое поле содержит:
    - **Источник файла** (source);
    - **Информация об авторстве и классификации**:
        - author (Автор);
        - date (Дата);
        - genre (Жанр);
    - **Тип текста** (text_type): например, кулинарный текст;
    - **Статистика текста**:
        - word_count (Количество слов);
        - char_count (Количество символов);
        - analysis_time_ms (Время анализа);
    - **Системная информация**:
        - created_at (Дата создания — временная метка DateTime).

В таблице **sentences** (Сегментация текста) хранятся:

- **Уникальный ID предложения** (id);
- **Ссылка на документ** (document_id): связь с таблицей документов;
- **Порядковый номер** (sentence_index): позиция предложения в тексте;
- **Текст предложения** (sentence_text);
- **Дата создания** (created_at).

В таблице **tokens** (Морфологический анализ/Лексемы) хранятся данные о каждом слове:

- **Уникальный ID токена** (id);
- **Ссылки на сущности** (document_id, sentence_id);
- **Позиционирование** (token_index, position);
- **Лингвистические атрибуты**:
    - token_text (Форма слова);
    - lemma (Начальная форма/Лемма);
    - **Часть речи**: код (pos) и название (pos_name);
    - **Морфологические признаки**:
        - Падеж (case, case_name);
        - Число (number, number_name);
        - Род (gender, gender_name);
        - Время глагола (tense);
        - Лицо (person);
        - Одушевлённость (animacy);
    - **Синтаксическая роль**: код (syntax_role) и название (syntax_role_name).

В таблице **syntax_relations** (Синтаксические связи) хранятся:

- **Уникальный ID связи** (id);
- **Ссылка на предложение** (sentence_id);
- **Участники связи**:
    - from_token_id (Токен-источник/Главное слово);

    - to_token_id (Токен-цель/Зависимое слово);
- **Тип связи** (relation_type, relation_name);
- **Описание связи** (description);
- **Дата создания** (created_at).

**Связи между таблицами:**

- Один документ (documents) имеет много предложений (sentences).
- Одно предложение (sentences) имеет много токенов (tokens).
- Токены (tokens) связаны между собой синтаксическими отношениями (syntax_relations) в пределах одного предложения.

# Структурная схема приложения

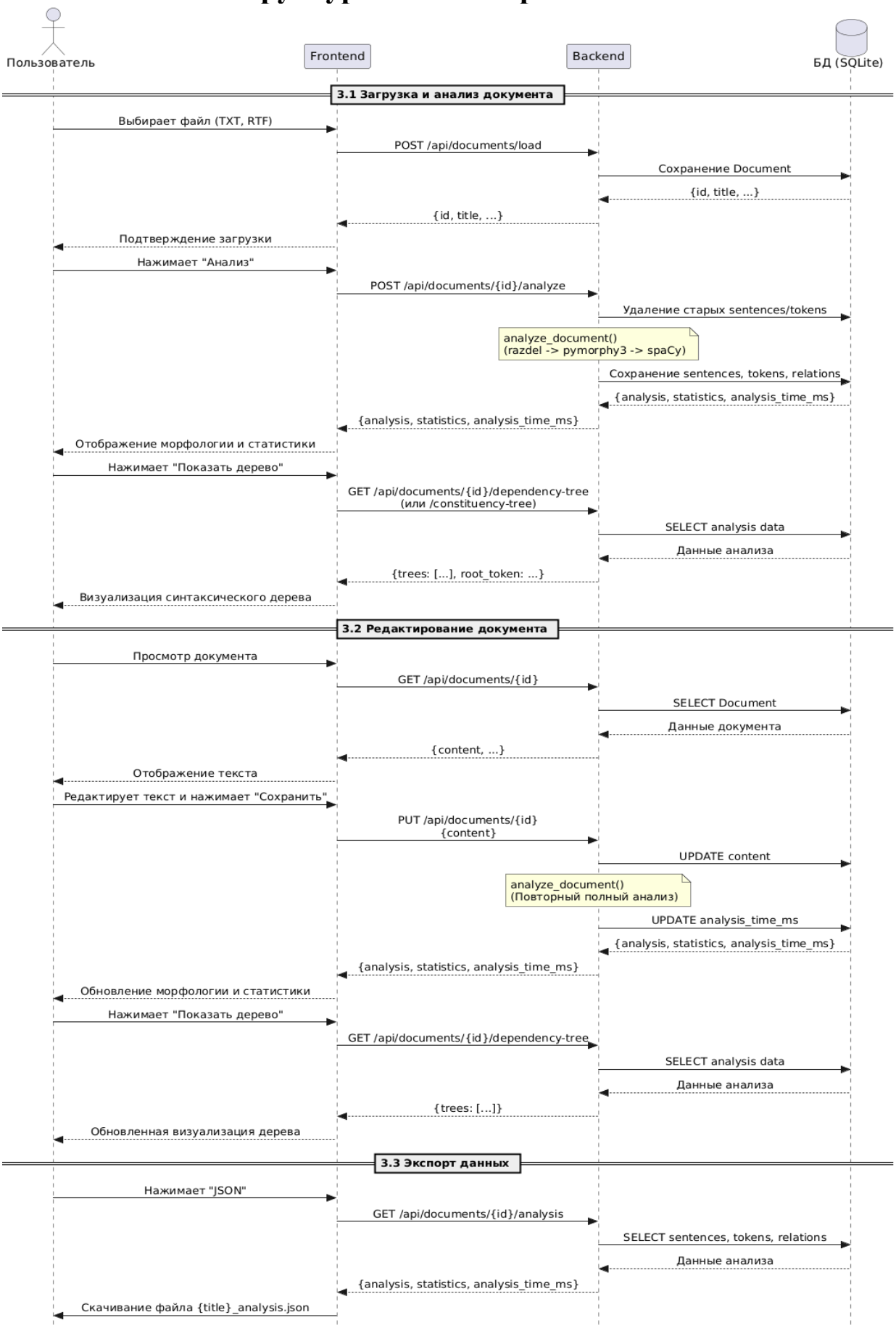


***Рисунок 3:*** *Структурная схема приложения*

## 1. Алгоритм работы программы

1. Начало.
2. Пользователь загружает текстовый документ (TXT, RTF) через веб-интерфейс.
3. Система извлекает текст, определяет метаданные и сохраняет документ в базу данных SQLite.
4. Программа выполняет комплексный лингвистический анализ: от токенизации до построения синтаксических деревьев.
5. Результаты анализа (морфологические признаки, синтаксические роли, иерархические связи) визуализируются на экране.
6. Пользователь взаимодействует с данными:
   * Редактирует текст документа (с автоматическим пересчетом анализа);
   * Просматривает статистику распределения частей речи и синтаксических ролей;
   * Анализирует структуру предложений через интерактивные деревья зависимостей и составляющих.
7. Конец.

## 2. Алгоритм морфологического и синтаксического анализа

1. Начало.
2. Система разбивает исходный текст на предложения и токены с помощью библиотеки razdel.
3. Для каждого токена с помощью pymorphy3 определяются: лемма, часть речи и грамматические признаки (падеж, число, род и др.).
4. На основе морфологических данных и эвристических правил определяются предварительные синтаксические роли слов.
5. Синтаксический этап:
   * С помощью модели spaCy строится дерево зависимостей (Dependency Tree) для определения иерархии «главное слово и зависимое».
   * На основе структуры предложения строится дерево составляющих (Constituency Tree) для выделения именных и глагольных групп.
6. Система вычисляет общую статистику по документу (количество слов, распределение POS-тегов).
7. Все полученные данные (токены, связи, деревья) сохраняются в реляционные таблицы базы данных.
8. Конец.

## 3. Алгоритм обработки изменений и актуализации данных

1. Начало.
2. Пользователь вносит изменения в текст документа через интерфейс редактора.
3. Система обновляет содержимое документа в базе данных и удаляет устаревшие результаты анализа для этого документа.

4. Запускается повторный цикл полного анализа (морфология синтаксис деревья).
5. Программа замеряет время выполнения анализа (analysis_time_ms) для оценки производительности.
6. Обновленные данные передаются на фронтенд в формате JSON для мгновенного обновления визуализации деревьев и статистики.
7. Конец.

## Тест производительности

Для оценки производительности приложения был проведён тест на семантико-морфологический анализ объёмом в среднем **200** слов. Среднее время обработки составило ~35 милисекунд на ПК HP Victus Pavilion 15, процессор: AMD Ryzen 5 8645HS, объём оперативной памяти: 16 ГБ.

**Результаты замеров:**

| Количество слов | Время обработки (мс) | мс/слово |
|---|---|---|
| 137 | 30 | 0.22 |
| 204 | 48 | 0.23 |
| 339 | 81 | 0.24 |

Основное время затрачивается на поиск.

## Вывод

Изучено назначение и функциональная структура лингвистического процессора для автоматического анализа ТЕЯ, а также практически закреплены навыки программирования при реализации алгоритмов обработки, хранения и систематизации лингвистических данных в рамках единой программной системы.